

Код сервисов и код скриптов проекта PyAnswer

Финальный комплект для сдачи практики

Проект: **PyAnswer**
Ветка: `main`
Commit: `cc88219998c0ee34a7d7fafd744b5bf79e7865a5`
Версия backend/compose: `0.7.0` ; версия frontend package: `0.1.0`
Tag: отсутствует
Дата: 16 июля 2026 года, Europe/Moscow

1. Назначение проекта

PyAnswer — локальная поисковая и RAG-система по вопросам о Python со Stack Overflow на русском. PostgreSQL хранит исходный корпус и операционные данные, Qdrant — восстанавливаемый sparse/dense индекс, Ollama — локальные embedding- и generation-модели. React-интерфейс обращается к FastAPI только через единый origin Nginx. Внешний облачный LLM API в фактической конфигурации не используется.

Комплект описывает состав исходного кода без переписывания бизнес-логики. Полные исходники предоставляются ссылкой на GitHub и безопасным архивом `PyAnswer_source.zip` ; огромные листинги в документ не включены.

2. Репозиторий, версия и состояние Git

Поле	Фактическое значение
GitHub	<code>https://github.com/4iriks/praktika</code>
origin	<code>https://github.com/4iriks/praktika.git</code>
Ветка	<code>main</code>
Commit	<code>cc88219998c0ee34a7d7fafd744b5bf79e7865a5</code>
Последний commit	Завершён седьмой этап: интеграция и подготовка PyAnswer к защите
Дата commit	<code>2026-07-16T22:07:50+03:00</code>
Release tag	отсутствует
Рабочее дерево до формирования комплекта	не чистое: существовали <code>untracked artifacts/</code> , <code>docs/final/presentation/</code> , <code>scripts/build_final_presentation.cjs</code> , <code>scripts/capture_final_presentation.mjs</code>

Существующие пользовательские файлы не изменялись. `origin` , теги и история Git не менялись; push не выполнялся.

3. Общая структура исходного кода

```
PyAnswer/
├── backend/
│   ├── app/
│   │   ├── api/routes/           # FastAPI routes
│   │   ├── schemas/             # Pydantic contracts
│   │   ├── services/            # прикладные сервисы
│   │   ├── db/repositories/     # SQLAlchemy repositories
│   │   ├── integrations/        # Qdrant, Ollama, reranker, Stack Exchange
│   │   ├── processing/          # очистка, canonicalization, чанкинг
│   │   ├── workers/             # ingestion и indexing runners
│   │   ├── scripts/             # служебные Python CLI
│   │   ├── main.py              # FastAPI entry point
│   │   ├── worker.py            # ingestion worker entry point
│   │   ├── indexer.py           # indexer entry point
│   │   └── reranker.py          # reranker FastAPI entry point
│   ├── alembic/versions/        # 6 миграций
│   ├── tests/                  # unit/integration tests
│   ├── Dockerfile
│   ├── Dockerfile.reranker
│   └── pyproject.toml
├── frontend/
│   ├── src/                    # React/TypeScript
│   ├── Dockerfile
│   ├── nginx.conf
│   ├── package.json
│   └── package-lock.json
├── scripts/                   # backup, restore-check, env, presentation helpers
├── evaluation/                # search/RAG datasets
├── e2e/                       # end-to-end test sources
├── config/models.yaml
├── docs/
├── compose.yaml
├── compose.gpu.yaml
├── Makefile
├── .env.example
└── README.md
```

Корневого `pyproject.toml` и корневого `package.json` нет: фактические manifests находятся в `backend/pyproject.toml` и `frontend/package.json`. Каталог `workers/` в корне отсутствует; `worker`-код расположен в `backend/app/workers/`, а точки входа — в `backend/app/worker.py` и `backend/app/indexer.py`.

Код сервисов

4. Таблица сервисов

В `compose.yaml` реализовано ровно восемь сервисов. Nginx является runtime-слоем контейнера `frontend`, а не отдельным compose-сервисом. Отдельного `migration/bootstrap` сервиса нет: одноразовые операции выполняются через образ `backend`.

Сервис	Назначение	Технология	Исходный код	Точка входа	Запуск
frontend	SPA, единый origin и reverse proxy	React 18, TypeScript, Vite, Nginx 1.27	frontend/src/ , frontend/nginx.conf	frontend/src/main.tsx ; Nginx	docker-compose up -d frontend
backend	HTTP API, auth/RBAC, search, RAG, editor/admin	FastAPI, SQLAlchemy, asyncpg	backend/app/	app.main:app	docker-compose up -d backend
postgres	Source of truth и очередь jobs	PostgreSQL 16	migrations/models в backend/	upstream postgres	docker-compose up -d postgres
qdrant	Sparse/dense производный индекс	Qdrant 1.18.2	client/schema в backend/app/integrations/qdrant/	upstream Qdrant	docker-compose up -d qdrant
ollama	Локальные embeddings и LLM generation	Ollama 0.32.0	adapters в backend/app/integrations/{embeddings, llm}/	upstream Ollama	docker-compose up -d ollama
worker	Ingestion из Stack Exchange	Python asyncio durable worker	backend/app/worker s/ , backend/app/worker .py	python -m app.worker	docker-compose up -d worker
indexer	Document/full reindex и публикация индекса	Python asyncio, Qdrant, Ollama	backend/app/worker s/indexer.py , backend/app/services/search_index.py	python -m app.indexer	docker-compose up -d indexer
reranker	Локальное переранжирование кандидатов	FastAPI, sentence-transformers, Qwen3 reranker	backend/app/reranker.py	app.reranker:app	docker-compose up -d reranker

5. Описание сервисов

5.1 frontend: React и Nginx

- Назначение и функции:** пользовательский поиск BM25/vector/hybrid, RAG-ответ со streaming, документы, история, сохранённое, профиль, editor/admin панели.
- Технология:** React 18.3.1, TypeScript 5.7.2, Vite 5.4.21; production-раздача Nginx 1.27-alpine.
- Код и entry point:** frontend/src/ , frontend/src/main.tsx , routes в frontend/src/routes/AppRoutes.tsx.
- Маршруты:** / , /search , /documents/:documentId , /login , /register , /profile , /history , /saved , /editor... , /admin... , /403 .
- Dockerfile/запуск/порт:** frontend/Dockerfile; nginx -g 'daemon off;'; host 127.0.0.1:\${FRONTEND_PORT:-8080} → container 80 .
- Зависимости:** backend должен быть healthy. /api/ проксируется на http://backend:8000 ; /api/ask/stream настроен без buffering для SSE.
- Healthcheck:** GET /healthz через wget ; фактически проверен ответ ok .
- Volumes:** нет.

- **Environment/build args:** `VITE_USE MOCKS`, `VITE_API_BASE_URL`, `FRONTEND_PORT`, `APP_VERSION`.
- **Безопасность:** браузер работает только по HTTP с Nginx/FastAPI и не подключается напрямую к PostgreSQL, Qdrant, Ollama или reranker. Серверный FastAPI выполняет auth/RBAC; frontend guards не являются границей безопасности.
- **Статус:** реализован; production image и healthcheck присутствуют.

5.2 backend : FastAPI API

- **Назначение:** auth, серверный RBAC `USER / EDITOR / ADMIN`, Search API, RAG API, editor/admin API, durable jobs, audit и healthchecks.
- **Структура:** routes — `backend/app/api/routes/`; schemas — `backend/app/schemas/`; services — `backend/app/services/`; repositories — `backend/app/db/repositories/`; integrations — `backend/app/integrations/`.
- **Entry point/Dockerfile:** `backend/app/main.py`, `uvicorn app.main:app --host 0.0.0.0 --port 8000`; `backend/Dockerfile`.
- **Порт:** host `127.0.0.1:${BACKEND_PORT:-8000}` → container `8000`; внутри frontend обращается к `backend:8000`.
- **API-группы:** `/api/auth`, `/api/users`, `/api/search`, `/api/ask`, `/api/ask/stream`, `/api/documents`, `/api/history`, `/api/saved`, `/api/feedback`, `/api/editor`, `/api/admin/*`, `/api/health/live`, `/api/health/ready`.
- **Auth/RBAC:** HttpOnly session cookies, CSRF для unsafe methods, Argon2 и permission dependencies на сервере; audit-события сохраняются для управляющих действий.
- **Зависимости:** в compose запуск зависит от healthy PostgreSQL; во время search/RAG также использует Qdrant, Ollama и reranker через внутренние Docker URL.
- **Healthcheck:** HTTP GET `/api/health/ready`; live и ready фактически ответили `ok/ready`.
- **Volumes:** нет.
- **Основные env:** `APP_ENV`, `APP_NAME`, `APP_VERSION`, `DEBUG`, `DATABASE_URL`, `FRONTEND_ORIGINS`, `SESSION_COOKIE_NAME`, `CSRF_COOKIE_NAME`, `SESSION_TTL_HOURS`, `REMEMBER_SESSION_TTL_DAYS`, `COOKIE_SECURE`, `COOKIE_SAMESITE`, `ARGON2_*`, `BOOTSTRAP_ADMIN_*`, `QDRANT_*`, `OLLAMA_BASE_URL`, `EMBEDDING_*`, `RERANKER_*`, `LLM_*`, `RAG_*`. Значения секретов не приводятся.
- **Статус:** реализован.

5.3 postgres : PostgreSQL

- **Роль:** единственный source of truth. Qdrant и модели считаются производными/восстанавливаемыми.
- **Данные:** users, roles, permissions, sessions; sources, documents, answers, tags, revisions, chunks; durable jobs, job events, worker instances, ingestion state/failures; search runs/index versions; history, saved items, RAG responses/sources, feedback и audit events.
- **Код схемы:** `backend/app/db/models/`, `backend/alembic/versions/`.
- **Image/entry point:** `postgres:16-alpine`, стандартный upstream entrypoint; отдельного Dockerfile нет.
- **Порт:** host `127.0.0.1:${POSTGRES_PORT:-5432}` → `5432`.
- **Healthcheck:** `pg_isready`; live container фактически healthy.
- **Volume:** `postgres_data:/var/lib/postgresql/data`.
- **Env:** `POSTGRES_DB`, `POSTGRES_USER`, `POSTGRES_PASSWORD`, `POSTGRES_INITDB_ARGS`; пароль не раскрывается.
- **Взаимодействие:** `backend/worker/indexer` используют SQLAlchemy + asyncpg; очередь jobs, lease и heartbeat хранятся в PostgreSQL.
- **Статус:** реализован; live `alembic_version` фактически `20260716_0006`.

5.4 qdrant : поисковый индекс

- **Назначение:** производный индекс chunks и retrieval для Search/RAG.
- **Схема:** named vectors `dense` и `sparse`; `dense` — cosine HNSW, 1024 dimensions, `m=16`, `ef_construct=100`; `sparse` — native `qdrant/bm25` с IDF.
- **Hybrid:** BM25 и dense candidates объединяются weighted RRF; опционально вызывается локальный reranker.

- **Blue-green:** физические collections строятся отдельно, валидируются, затем атомарно переключается alias `pyanswer_chunks_current`; PostgreSQL хранит версии индекса.
- **Код:** `backend/app/integrations/qdrant/`, `backend/app/services/search_index.py`, `backend/app/search/`.
- **Image/порт:** `qdrant/qdrant:v1.18.2`; `host 127.0.0.1:${QDRANT_PORT:-6333} → 6333`; отдельного Dockerfile нет.
- **Healthcheck:** `TCP /dev/tcp/127.0.0.1/6333`; `live /healthz` фактически успешен.
- **Volume:** `qdrant_data:/qdrant/storage`.
- **Env:** `QDRANT_URL`, `QDRANT_ALIAS`, `QDRANT_COLLECTION_PREFIX`, `QDRANT_TIMEOUT_SECONDS`, `QDRANT_MAX_RETRIES`, `SPARSE_*`, `INDEX_*`, `SEARCH_*`.
- **Зависимости/связи:** indexer пишет, backend читает; браузер доступа не имеет.
- **Статус:** реализован.

5.5 ollama: локальные модели

- **Назначение:** локальная LLM generation и embeddings без внешнего облачного LLM API.
- **Модели в compose:** generation `qwen3:8b`; embedding `qwen3-embedding:0.6b`, 1024 dimensions. Фактические имена задаются env.
- **Код adapters:** `backend/app/integrations/llm/ollama.py`, `backend/app/integrations/embeddings/ollama.py`.
- **Image/порт:** `ollama/ollama:0.32.0`; `internal 11434`, наружу в compose не публикуется.
- **Healthcheck:** `ollama list`; `live CPU container` фактически healthy.
- **Volume:** `ollama_models:/root/.ollama`.
- **Env:** `OLLAMA_BASE_URL`, `OLLAMA_MAX_LOADED_MODELS`, `OLLAMA_NUM_PARALLEL`, `EMBEDDING_*`, `LLM_*`; optional NVIDIA runtime в `compose.gpu.yml`.
- **Зависимости/связи:** indexer получает embeddings; backend вызывает generation; браузер прямого доступа не имеет.
- **Статус:** реализован; скачивание моделей является отдельной ресурсоёмкой командой и при формировании комплекта не запускалось.

5.6 worker: ingestion worker

- **Назначение:** initial/incremental Stack Exchange sync, получение вопросов/ответов, очистка HTML, selection, identity/content deduplication, revisions и deterministic chunking с сохранением в PostgreSQL.
- **Код/entry point:** `backend/app/worker.py`, `backend/app/workers/runner.py`, `backend/app/workers/source_sync.py`, `backend/app/services/document_ingestion.py`, `backend/app/processing/`; `python -m app.worker`.
- **Dockerfile/порт:** общий `backend/Dockerfile`; собственного опубликованного порта нет.
- **Durability:** jobs в PostgreSQL, lease, heartbeat, checkpoint после committed batch, retry до `WORKER_MAX_ATTEMPTS`, cancellation и graceful shutdown.
- **Зависимости:** healthy PostgreSQL; внешний HTTPS Stack Exchange API.
- **Healthcheck:** `python -m app.scripts.worker_health --capability source_sync`.
- **Volumes:** нет.
- **Env:** `DATABASE_URL`, `STACK_EXCHANGE_*`, `STACKEXCHANGE_*`, `WORKER_POLL_INTERVAL_SECONDS`, `WORKER_LEASE_SECONDS`, `WORKER_HEARTBEAT_SECONDS`, `WORKER_MAX_ATTEMPTS`, `WORKER_SHUTDOWN_TIMEOUT_SECONDS`.
- **Статус:** код реализован. Один ранее запущенный live-контейнер имел transient status `unhealthy`; это отражено как эксплуатационное предупреждение, а не как отсутствие кода.

5.7 indexer: indexing worker

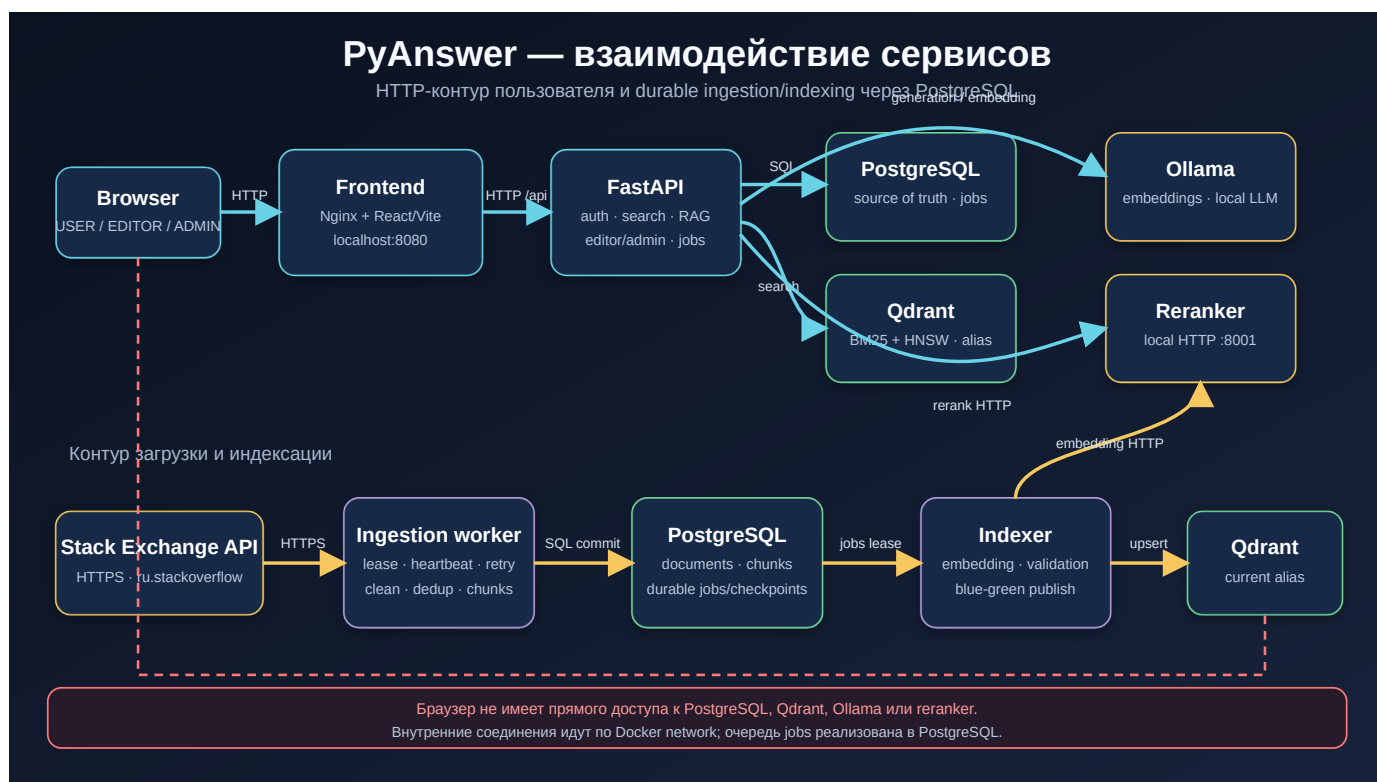
- **Назначение:** `DOCUMENT_REINDEX`, `FULL_REINDEX`, validation/cleanup, embedding, sparse vectors, Qdrant upsert и blue-green alias switch.
- **Код/entry point:** `backend/app/indexer.py`, `backend/app/workers/indexer.py`, `backend/app/services/search_index.py`; `python -m app.indexer`.
- **Dockerfile/порт:** общий `backend/Dockerfile`; наружного порта нет.

- **Durability:** очередь PostgreSQL, lease/heartbeat/retry/cancellation; полная переиндексация защищена `CONFIRM_INDEX_FULL=YES` в CLI.
- **Зависимости:** healthy PostgreSQL, Qdrant и Ollama; reranker не нужен для построения индекса.
- **Healthcheck:** `python -m app.scripts.worker_health --capability search_index`.
- **Volumes:** нет; пишет в Qdrant volume через API.
- **Env:** `DATABASE_URL`, `QDRANT_*`, `OLLAMA_BASE_URL`, `EMBEDDING_*`, `SPARSE_*`, `INDEX_*`, `WORKER_*`.
- **Статус:** код реализован. Полная переиндексация при подготовке отчёта не запускалась.

5.8 reranker : локальный reranker API

- **Назначение:** переранжирование до 8 passage-кандидатов; endpoints `GET /health/live`, `GET /health/ready`, `POST /rerank`.
- **Технология/модель:** FastAPI, sentence-transformers CrossEncoder, Qwen/Qwen3-Reranker-0.6B с закреплённой revision.
- **Код/entry point/Dockerfile:** `backend/app/reranker.py`; `uvicorn app.reranker:app --port 8001`; `backend/Dockerfile.reranker`.
- **Порт:** internal 8001, наружу не публикуется.
- **Зависимости:** compose ожидает PostgreSQL health; backend обращается к `http://reranker:8001`. Модель должна быть локально загружена.
- **Healthcheck:** `GET /health/ready`; live container фактически healthy.
- **Volume:** `reranker_models:/models/huggingface`.
- **Env:** `RERANKER_MODEL`, `RERANKER_MODEL_REVISION`, `RERANKER_DEVICE`, `RERANKER_BATCH_SIZE`, `RERANKER_MAX_LENGTH`, `RERANKER_TOP_N`, `HF_HOME`.
- **Статус:** реализован.

6. Связи между сервисами



Публичны только loopback-порты frontend/backend/PostgreSQL/Qdrant, причём пользовательский сценарий идёт через frontend. Внутри Docker network используются DNS-имена `backend`, `postgres`, `qdrant`, `ollama`, `reranker`. Jobs не передаются через отдельный broker: backend/CLI создают durable rows в PostgreSQL, worker/indexer получают lease и обновляют heartbeat.

Код скриптов

7. Состав служебного кода

Учтён **91 объект**: 18 backend Python CLI, 7 корневых скриптов, 6 Alembic migrations, 52 Makefile targets и 8 npm scripts. Полный компактный перечень находится в [Перечень_скриптов.md](#) ; ниже приведены исполняемые скрипты и их свойства.

7.1 Таблица скриптов

Скрипт/команда	Путь	Назначение	Пример запуска	Результат
Alembic upgrade	backend/alembic/versions/*.py	применить 6 ревизий схемы	docker-compose run --rm backend alembic upgrade head	schema revision 20260716_0006
bootstrap	backend/app/scripts/bootstrap.py	поли/permissions/bootstrap admin	docker-compose run --rm backend python -m app.scripts.bootstrap	записи и log
seed demo	backend/app/scripts/seed_demo.py	demo data	docker-compose run --rm -e SEED_DEMO_DATA=true backend python -m app.scripts.seed_demo	demo rows
enqueue sync	backend/app/scripts/enqueue_sync.py	SOURCE_SYNC job	... enqueue_sync <SOURCE_ID> --mode INITIAL --max-pages 2	job UUID
ingestion report	backend/app/scripts/ingestion_report.py	состояние ingestion	docker-compose run --rm worker python -m app.scripts.ingestion_report	JSON stdout
full reindex	backend/app/scripts/enqueue_full_reindex.py	FULL_REINDEX job	CONFIRM_INDEX_FULL=YES make index-full	job UUID
worker health	backend/app/scripts/worker_health.py	heartbeat/capability	... worker_health --capability source_sync	JSON + exit code
Stack Exchange smoke	backend/app/scripts/live_stackexchange_smoke.py	небольшой live source test	RUN_LIVE_STACKEXCHANGE_SMOKE=1 ...	items/quota
pull Ollama models	backend/app/scripts/pull_models.py	скачать embedding/LLM	make models-pull	installed JSON
pull reranker	backend/app/scripts/pull_reranker.py	скачать pinned snapshot	make reranker-model-pull	model path JSON
local model smoke	backend/app/scripts/live_local_models_smoke.py	live Search/RAG smoke	RUN_LIVE_LOCAL_MODELS=1 make local-model-smoke	checks stdout
retrieval evaluation	backend/app/scripts/evaluate_retrieval.py	BM25/vector/hybrid metrics	make search-evaluate	JSON + Markdown
RAG evaluation	backend/app/scripts/evaluate_rag.py	source/citation/key-point metrics	make rag-evaluate	JSON + Markdown
corpus manifest	backend/app/scripts/corpus_manifest.py	corpus counts/manifest	make corpus-manifest	artifacts/corpus-manifest.{json,md}
data consistency	backend/app/scripts/data_consistency_check.py	relational checks	make data-check	report + exit code
index consistency	backend/app/scripts/index_consistency_check.py	DB/Qdrant consistency	make index-check	report + exit code
disk report	backend/app/scripts/disk_report.py	disk budget	make disk-report	report

Скрипт/команда	Путь	Назначение	Пример запуска	Результат
acceptance check	backend/app/scripts/acceptance_check.py	system acceptance	make demo-check	report + exit code
release manifest	backend/app/scripts/release_manifest.py	version/checksum manifest	make release-manifest	report
backup	scripts/backup.py , scripts/backup.sh	pg_dump + optional Qdrant snapshot	make backup	timestamped backup + checksums
restore check	scripts/restore_check.py	non-destructive validation	make restore-check BACKUP_PATH=backups/<timestamp>	restore-check.json
restore guard	scripts/restore.sh	запрет overwrite основной среды	CONFIRM_RESTORE=YES scripts/restore.sh	намеренный отказ, no restore
local env	scripts/generate-local-env.py	создать .env с local secrets	python3 scripts/generate-local-env.py	mode 0600 .env
presentation build	scripts/build_final_presentation.cjs	сборка PPTX/PDF отчётности	node scripts/build_final_presentation.cjs	presentation files
presentation capture	scripts/capture_final_presentation.mjs	live screenshots	node scripts/capture_final_presentation.mjs --home-only	PNG/manifest

7.2 Параметры, безопасность повторного запуска и ошибки

Скрипт	Входы/env	Повторный запуск	Изменяет данные / подтверждение	Типовые ошибки
migrations	DATABASE_URL ; revision	да, upgrade head повторяем	схема БД; отдельного confirm нет	БД недоступна, конфликт migration
bootstrap	DATABASE_URL , BOOTSTRAP_ADMIN_*	спроектирован идемпотентно	БД; confirm нет	migrations не применены, invalid env
seed_demo	SEED_DEMO_DATA=true	спроектирован идемпотентно	БД; явный env-флаг обязателен	флаг отсутствует, БД недоступна
enqueue_sync	source_id , --mode , --max-documents , --max-pages , --dry-run	создаёт новую job; duplicate защищён	БД; uncapped требует CONFIRM_FULL_SYNC=YES	bad UUID/limits, source disabled/not found, active duplicate
enqueue_full_reindex	CONFIRM_INDEX_FULL=YES	duplicate защищён	БД/jobs; подтверждение обязательно	active full reindex, БД недоступна
report/check scripts	DATABASE_URL , REPO_ROOT , ARTIFACTS_DIR , provider URL env	повторяемы; перезаписывают reports	только report files, кроме чтения live data	service unavailable, inconsistency, permission
evaluations	--base-url , --dataset , --output , retrieval --split	повторяемы при новом output	создают JSON/MD; RAG вызывает generation и пишет историю	unreviewed/empty dataset, API/model timeout
pull_models	OLLAMA_BASE_URL , model env, --embedding-only/--llm-only	Ollama повторно сверяет модель	volume models; ресурсоёмко, ручной запуск	network/disk/Ollama error
pull_reranker	RERANKER_MODEL , pinned revision, HF_HOME	cache-aware	model volume; ресурсоёмко	network/disk/revision unavailable
backup	--postgres-container , --qdrant-url , --output	только в новый каталог	создаёт backup/snapshot; без confirm	container missing, target exists, pg_dump/snapshot error
restore_check	positional backup path	да; обновляет report	не трогает main volumes	checksum/dump invalid, Docker unavailable
restore.sh	CONFIRM_RESTORE=YES	всегда безопасно отказывает	destructive restore не реализован	exit 1 by design

Скрипт	Входы/env	Повторный запуск	Изменяет данные / подтверждение	Типовые ошибки
generate-local-env	--path, --force, --reuse-postgres-environment	без --force не перезаписывает	создаёт secret file; force требует осознанности	missing source env, permissions
presentation helpers	live URL/files; --home-only у capture	перезаписывают generated report assets	только отчётность; no confirm	browser/dependency/live UI unavailable

8. Классификация скриптов

- 1. **Инициализация:** Alembic migrations; bootstrap; seed_demo; generate-local-env.py.
- 2. **Сбор данных:** live_stackexchange_smoke; enqueue_sync в INITIAL/INCREMENTAL; worker; ingestion_report. Полный импорт защищён confirm-флагами.
- 3. **Обработка:** очистка HTML — backend/app/processing/html.py; canonicalization/dedup — canonical.py, normalization.py, selection.py, document_ingestion.py; чанкинг — chunking.py. Это внутренние модули ingestion worker, отдельных CLI для них нет.
- 4. **Индексация:** document reindex — POST /api/editor/documents/{document_id}/reindex; full reindex — enqueue_full_reindex; validation/cleanup и blue-green alias switch выполняются indexer/service search_index.py. Отдельного ручного CLI для прямого alias switch нет, что безопаснее.
- 5. **Проверка:** unit/integration tests, npm tests/typecheck/lint, compose validation, worker_health, consistency scripts, acceptance, retrieval/RAG evaluation, corpus/release manifests.
- 6. **Эксплуатация:** Make targets для up/down/status/logs; backup, restore-check, disk report, model status/pull и safe cleanup.

Отдельные команды «создать администратора», «очистить HTML», «deduplicate», «chunk document», «переключить alias» **не найдены**. Администратор создаётся bootstrap-процессом; обработка встроена в ingestion; alias переключает только валидированный indexer pipeline. Это фактическое отличие от возможных более ранних описаний/чек-листов.

9. Команды запуска

```
# Первый безопасный запуск; без полного импорта и без автозагрузки больших моделей
make first-run

# Обычный запуск/статус/остановка
make demo-up
make demo-status
make demo-logs
make demo-down

# Backend и migrations
make backend-up
make backend-migrate

# Ограниченный ingestion
make sync-smoke SOURCE_ID=<uuid>
make sync-incremental SOURCE_ID=<uuid>

# Полная индексация — только явное подтверждение
CONFIRM_INDEX_FULL=YES make index-full
```

Полный копируемый список приведён в Команды_запуска.md.

10. Команды проверки и фактические статусы

Проверка	Статус	Факт
Git URL/branch/commit/tag/status	VERIFIED	команды Git выполнены
docker-compose config --quiet	VERIFIED	exit 0
CPU + GPU compose config	VERIFIED	exit 0
Python syntax	VERIFIED	AST parse проекта выполнен
Импорт 18 backend CLI	VERIFIED	все 18 импортированы без bytecode writes
--help для 5 backend и 3 root CLI	VERIFIED	exit 0
Node syntax двух presentation helpers	VERIFIED	node --check, exit 0
52 Make targets	EXISTS, NOT RUN	все прошли make -n; операции не выполнялись
Alembic chain/head	VERIFIED	6 revisions; head 20260716_0006
Live DB migration row	VERIFIED	20260716_0006 через read-only SQL
frontend/backend/Qdrant HTTP health	VERIFIED	ответы успешны
full import/full reindex/model pulls	REQUIRES CONFIRMATION	намеренно не запускались
evaluation/backup/restore-check	REQUIRES LIVE SERVICES	не запускались в рамках сборки комплекта
alembic current в ранее запущенном custom backend container	REQUIRES LIVE SERVICES	container был настроен на недоступный localhost:5432; revision отдельно подтверждена в PostgreSQL

Статусы и причины подробно зафиксированы в `Проверка_команд.md`.

11. Безопасность

- Архив формируется из tracked source плюс два существующих untracked script-helper и созданная документация.
- `.env`, `.git`, реальные secrets, credentials, cookies/tokens, backups, logs, volumes, model stores, raw API archives, caches, virtualenv, `node_modules`, `dist`, coverage и Playwright outputs исключены.
- Значения `POSTGRES_PASSWORD`, `BOOTSTRAP_ADMIN_PASSWORD`, `STACKEXCHANGE_KEY` и иные секреты не выводились и не документировались.
- В архиве допускаются только конфигурационные примеры `.env.example` с безопасными placeholder/пустыми значениями.
- Архив проверяется `unzip -t`, отдельным denylist по путям, размеру и текстовым secret patterns; создаётся SHA-256.
- Destructive reset, удаление volumes, restore поверх основной БД, полный import и полный reindex не выполнялись.

12. Состав архива

`PyAnswer_source.zip` содержит корень `PyAnswer/`: frontend source/manifests/lock-file; backend source, tests, migrations и Dockerfiles; root scripts; compose files; Makefile; configuration examples; evaluation/e2e sources; README и docs, включая этот комплект. `.git` не включён.

13. Результаты проверки

- Compose base/GPU конфигурации валидны.
- Восьми фактическим сервисам соответствуют compose entries и/или реальные entry points.
- Все 18 backend CLI импортированы; 52 Make targets фактически обнаружены и dry-run проверены.

- Migration head и live DB revision совпадают: 20260716_0006 .
- Рабочие HTTP health endpoints frontend/backend/Qdrant ответили успешно.
- Один ранее запущенный ingestion worker и один custom GPU indexer имели unhealthy ; это live-операционное состояние, код и compose healthchecks присутствуют.
- Итоговые проверки ZIP/PDF/PNG и checksum приведены в Проверка_комплекта.md .

14. Расхождения документации и кода

- В app.main description сохранилась фраза «RAG появится в 6.3», хотя RAG routes/services/migration 0006 уже реализованы. В отчёте использован фактический код.
- README говорит о browser-only unified origin; compose одновременно публикует backend/PostgreSQL/Qdrant только на loopback для локальной диагностики. Прямого browser-вызова этих сервисов frontend-код не делает.
- Отдельных migration/bootstrap, Nginx, document-processing Docker-сервисов нет; это соответственно one-shot backend commands, слой frontend image и код worker pipeline.
- Корневые presentation helpers существуют, но были untracked до сборки; они отмечены как отчётные инструменты, не как бизнес-сервисы.

15. Заключение

Два обязательных пункта — **«Код сервисов»** и **«Код скриптов»** — представлены отдельными явно выделенными разделами. Комплект описывает только существующие файлы, сервисы и команды; исходный код не переписывался. Доступ к коду обеспечен GitHub-ссылкой и проверенным локальным ZIP с checksum.